Proposed change [A93], congestion is measured as the time picks stand still at each shelf

13 November 2026, version 3 (plainer words, the reason for keeping demand on the nearest shelf)

How to read this

Part 1 says what we found and what is proposed. Part 2 shows the thesis text that would change. Green is new text, ~~struck~~ text goes, black text stays, and italic text is a note on what to draw or add. A removed equation is grey and marked "removed". Part 3 has the questions. Nothing is changed in the thesis or the code yet.

Part 1. What we found

The old signal did not work

Congestion-aware storage counts, for each edge, how often agents passed it times how often they waited to get onto it.

- **The number is far too small.** It is about 10^{-4} per edge, while a trip is about 29 steps. So it never changed which shelf was chosen. This is why β from 0 to 4 made almost no difference.
- **It jumps around.** Only 18 to 34 of 782 edges have any wait in a storage period. The order of the shelves it gives changes almost completely from one period to the next.
- **It looks in the wrong place.** With Token Passing a pick stands still only 1 to 2.5 steps. Most of the time is spent waiting for a free agent, and that wait is not on any edge.

We tried several other edge signals with Token Passing on 10 seeds. None of them was better than demand storage.

The learned controllers do get stuck

The learned agents stand still much longer than with Token Passing, and more the bigger the fleet.

	steps standing still per pick	Token Passing
8 agents, decentralised	about 21	1 to 2.5
16 agents, section	about 32	1 to 2.5
16 agents, decentralised	about 38	1 to 2.5
32 agents, decentralised	74 to 85	1 to 2.5

The agents stand still in queues, when two agents face each other, and at the workstation lock of [A91]. Counting waits on edges misses most of this.

The new signal

When a pick is done, we count the steps its agent stood still and note them on the shelf it came from. The signal of a shelf is the average over its recent picks.

- It is counted in steps, like the trip. So $\beta = 1$ means one step standing still counts the same as one step of travel.
- It catches every reason for standing still.
- It belongs to a shelf, which is exactly what storage chooses.

At 8 agents, shelves differ a lot, from about 3 to 68 steps per pick, and the order of the shelves stays almost the same from one period to the next.

The signal is used in two places. When stock arrives, it goes where picks rarely stand still. And when a product is on several shelves, a pick takes it from the shelf with the lowest distance plus standing still, not just the nearest one. The second happens for every order, so the loop now also reacts quickly.

The test

We used policies that are already trained under demand storage, the best saved policy of each run. Nothing was retrained. We compared demand storage as it is with the signal used when stock arrives, when picking, and in both. 5 seeds, 4000 steps, of which the first 1000 are not counted. The test also averaged the demand numbers over periods (question 2).

		steps still per pick	less still than today	tasks done	as many tasks or more	keeps up
8 agents decentralised	today	21.0		339		4/5
	when stock arrives	18.6	3/5	339	3/5	4/5
	when picking	20.8	3/5	334	3/5	4/5
	both	**17.3**	**5/5**	340	3/5	4/5
16 agents section	today	32.5		361		0/5
	when stock arrives	29.4	4/5	385	4/5	0/5
	when picking	31.6	4/5	374	3/5	0/5
	both	**29.1**	**5/5**	**384**	4/5	0/5

Used in both places, the signal cut the time standing still on every seed. It was 18 percent less at 8 agents and 10 percent less at 16. At 16 agents the agents also did 6 percent more tasks. At 8 agents a pick took 244 steps instead of 294. This is the first congestion signal that helps.

Two runs tell us nothing. The 16-agent decentralised controller does only 115 to 150 tasks, where the section controller does 360 to 385. The 32-agent controller does 4 to 9 tasks in 3000 steps. Both get stuck over a long run. You said the 16-agent decentralised controller needs messages to work, so we test it with messages (question 5).

What this test is not

It is a short test on 5 seeds, with policies that never trained with the signal. It shows the signal is worth keeping. The full test in question 5 gives the result. Sending orders to the least busy workstation and moving old stock were tried the same day. They are not part of this change (question 7).

Part 2. The text

2.1 Problem Formulation, "What the rule knows"

~~The rule only knows the last storage period. It reads three averages per step over that period:~~ The rule only knows the recent past. It reads two numbers:

- $\hat{\rho}_t(k)$, how often SKU k was ordered per step in the last storage period
- ~~$\hat{\mu}_t(e)$, how often agents crossed edge e~~
- ~~$\hat{w}_t(e)$, how many steps agents waited to get onto edge e~~
- $\hat{\omega}_t(v)$, how many steps recent picks from shelf v stood still, on average

Demand storage uses only the first, the orders. Congestion-aware storage uses ~~all three. The last two come from how the agents moved, and show which aisles were crowded. An edge counts as crowded when many agents cross it and they often wait to get onto it.~~ both. The second shows where agents lose time because of each other. It may be a queue in an aisle, two agents facing each other, or a queue at a workstation.

In `eq:storageupdate` the inputs change from $\hat{\rho}_t, \hat{\mu}_t, \hat{w}_t$ (removed) to $\hat{\rho}_t, \hat{\omega}_t$.

2.2 Problem Formulation, "What the rule decides"

Putting every popular SKU on the best shelves would send all traffic down the same few aisles. Congestion-aware storage therefore also avoids shelves ~~whose trip runs through crowded aisles.~~ where picks often stand still.

...

The crowding cost is this thesis's own, because the classic problem has no traffic. It adds up, over the same units, ~~the crowding $\hat{\mu}_t(e)\,\hat{w}_t(e)$ on every edge e of the trip through v,~~ how often their SKU is ordered times how long picks stand still at their shelf. Every pick from that shelf meets the wait, so a popular SKU pays it more often.

$$K = \sum_{k} \sum_{v} \Big(\sum_{e \in \mathrm{path}(v)} \hat{\mu}_t(e)\,\hat{w}_t(e) \Big)\, x_t^{\mathrm{new}}(k, v) \qquad \text{removed}$$

$$K = \sum_{k \in \mathcal{K}} \sum_{v \in V_{\mathrm{str}}} \hat{\rho}_t(k)\,\hat{\omega}_t(v)\, x_t^{\mathrm{new}}(k, v).$$

The rule aims for a placement with a small value of $D + \beta K$, where β sets how much ~~crowding~~ standing still counts against distance. Both count steps, so with $\beta = 1$ a step standing still counts as a step of travel.

2.3 Problem Formulation, the pickup rule

A line takes its units from the shelf nearest its workstation first. If that shelf holds enough, the line is one task. ... This is the *pickup rule*.

Under congestion-aware storage a line chooses differently. For each shelf it adds the distance to the workstation and the time picks there stand still, $\hat{\omega}_t(v)$. It takes from the shelf with the smallest sum first. Say tea is on A and C. Picks from A have stood still 5 steps on average and picks from C 1 step. Then the line goes to C if C is at most 3 steps farther away.

Fixed and demand storage keep taking the nearest shelf. Demand storage knows only the orders, so letting it pick by standing still would make it read the congestion too. The gap between demand and congestion-aware storage would then no longer show what reading congestion adds. The nearest shelf first is also the plain rule a warehouse uses when it knows nothing about traffic. So only the rule that closes the loop changes how lines pick.

This choice is made for every order line, while stock only arrives every Δ steps. So congestion-aware storage reacts both slowly and quickly.

Appendix C, one line after eq:pickuprule: the shelves $v_1, v_2, \ldots$ that hold free units of k are sorted by $d_G(v_i, g)$, or by $d_G(v_i, g) + \hat{\omega}_t(v_i)$ under congestion-aware storage, and by vertex number when equal.

2.4 Problem Formulation, the loop

Backward, routing shapes storage. ~~The traffic the agents create is measured in the estimates $\hat{\mu}_t$ and $\hat{w}_t$, and at the next epoch the storage update uses them to decide where items go.~~ How long agents stand still is measured at each shelf in $\hat{\omega}_t$. At the next storage update it decides where new stock goes. Every new order line also uses it to choose the shelf it picks from. With fixed storage only the forward path exists. The backward path is what congestion-aware storage adds, because only that rule reads the ~~traffic.~~ time agents stand still.

Figure loop.pdf, panel (b): the backward arrow says $\hat{\omega}_t$ instead of $\hat{\mu}_t, \hat{w}_t$, and a short second arrow goes from $\hat{\omega}_t$ to the pickup of each task. Caption: "The two-timescale loop under fixed (a) and congestion-aware storage (b). Demand storage reads only $\hat{\rho}_t$, so standing still never reaches it."

2.5 Implementation, "Where arriving stock goes"

Congestion-aware storage (F_{cng}) adds ~~crowding~~ standing still to the trip. Take two shelves with the same trip distance, ~~one reached through a quiet aisle and one through an aisle where agents often queue.~~ one where picks rarely stand still and one where they often do. Demand storage treats them as equal, and congestion-aware storage prefers the quiet one. Formally, the *congestion-aware score* of SKU k at shelf v is

$$\text{score}(k, v) = \hat{\rho}_t(k) \cdot d_t^{\text{trip}}(k, v) + \beta \sum_{e \in \text{path}(v)} \hat{\mu}_t(e)\, \hat{w}_t(e) \qquad \text{removed}$$

$$\text{score}(k, v) = \hat{\rho}_t(k) \left(d_t^{\text{trip}}(k, v) + \beta\, \hat{\omega}_t(v) \right),$$

~~where path(v) is the trip through v.~~ The weight $\beta \geq 0$ sets how much ~~crowding~~ standing still counts against distance.

~~The estimates are the per step rates of eq:estimates, taken over the last storage period only.~~ The demand number is the rate per step of eq:estimates over the last storage period. Counting per step keeps β comparable across the sweep of the storage period Δ. If no order was released in a period, nothing was picked and nothing arrives.

The time standing still comes from the picks done in the period. For each, we count the steps its agent stood still while serving it. This is the blocked part of its service time in eq:split. The average over a shelf's picks is the new reading for that shelf. A shelf gets only a few picks in one period, about eight at 8 agents, so one reading alone jumps around. So the number keeps four fifths of its old value and adds one fifth of the new reading. A shelf with no pick in the period keeps its old value. A shelf's first reading is used as it is. With this, the order of the shelves hardly changes from one period to the next.

~~A wait is counted on the edge the agent is waiting to enter, the next edge of its shortest route to its goal. It counts whenever an agent with a task stands still, whether it chose to wait or conflict resolution stopped it. Counting only the stops would show no waiting for Token Passing, which never collides but waits on purpose to let others pass. An agent without a task, or already at its goal, adds no wait.~~

2.6 Appendix C, eq:estimates

$$\hat{\rho}_t(k) = \tfrac{1}{\Delta}\,\#\{\text{order lines for } k \text{ released in the window}\},$$

$$\hat{\mu}_t(e) = \ldots, \quad \hat{w}_t(e) = \ldots \qquad \text{removed}$$

$$\hat{\omega}_t(v) = \tfrac{4}{5}\,\hat{\omega}_{t-\Delta}(v) + \tfrac{1}{5}\,\frac{1}{|P_t(v)|} \sum_{j \in P_t(v)} \sum_{t'=t_j^{\text{assigned}}}^{t_j^{\text{finish}}-1} \omega_j(t'),$$

where $P_t(v)$ are the picks from shelf v done in the window. If there are none, $\hat{\omega}_t(v) = \hat{\omega}_{t-\Delta}(v)$. A shelf's first reading is used as it is.

2.7 Appendix G, "Calibration: the congestion weight"

The congestion weight is $\beta = 1$. Both terms of the score count steps, ~~the expected travel and the expected waiting, so a step of waiting weighs the same as a step of travel.~~ travel and standing still, so a step standing still counts the same as a step of travel.

The rule first counted, for each edge, passes times waits. With Token Passing a weight from 0 to 4 changed the pick service time by at most 2 percent. The number was only about 10^{-4} per edge, and a pick stood still only 1 to 2.5 steps. Other edge signals did no better on 10 seeds. The time picks stand still at each shelf took its place [A93].

The learned controllers stand still much longer. With them the new signal cut the time standing still on every seed at 8 agents, and at 16 agents with the section controller, by 18 and 10 percent. *Table from Part 1, with the final numbers from the full test of question 5.*

~~The first sweep, on the first version of the warehouse graph, … that wait lies on no edge the weight can see.~~ *The three old paragraphs go, and their figure app_storage_beta.png too.*

2.8 Notation table and parameters

- ~~$\hat{\mu}_t(e)$, traffic estimate~~ and ~~$\hat{w}_t(e)$, waiting estimate~~ go.
- $\hat{\omega}_t(v)$, time standing still at a shelf. The average steps recent picks from shelf v stood still, kept over storage periods. Used by F and, under congestion-aware storage, by the pickup rule.
- β, congestion weight. How much the ~~crowding~~ standing still cost K counts against the slotting cost D.
- Parameters table, the reason for $\beta = 1$: ~~a step of waiting weighs as a step of travel~~ a step standing still counts as a step of travel. New row: the newest period counts one fifth in $\hat{\omega}_t$.

Part 3. Questions

1. **The new pick choice only for congestion-aware storage?** Decided yes (13 November). The reason is in 2.3. Demand storage knows only the orders, so the gap to congestion-aware storage must come from reading congestion alone. The main result, demand against fixed, stays the same and needs no new runs.
2. **Average the demand numbers too?** The test also averaged $\hat{\rho}_t$ over periods. To match the test, congestion-aware storage should do the same. But then the difference to demand storage comes from two changes. With Token Passing the averaging alone was 1.5 percent faster (8 of 10 seeds, $p = 0.08$). The other choice is to test again without it.
3. **One fifth for the newest period.** It was picked, not tuned. Keep it, or try a few values in the full test?
4. **Remove $\hat{\mu}_t$ and $\hat{w}_t$ completely?** I suggest yes, and also the paragraph on which edge a wait is counted on ([A36]). Nothing else uses them.
5. **The full test.** 20 test seeds, 10000 steps with the first 5000 not counted, the best saved policy of each run, demand against congestion-aware storage. Runs: 8 agents decentralised, 16 agents section, and 16 agents decentralised with messages. The 32-agent runs wait for a policy that does not get stuck. Retrain under the new rule later, or not at all?
6. **The name.** The rule now also chooses where to pick. Keep "congestion-aware storage", or say once that it also picks?
7. **Least busy workstation and moving old stock.** The least busy workstation made Token Passing about 28 percent faster on two seeds but did nothing for the learned controller. Moving old stock made Token Passing slower. Leave both out of [A93]?